

Assignment No: 7

Q1 Find first Occurrence: (return index of first occurrence target)

```

psvm() {
    int index = 0;
    int[] arr = {5, 2, 3, 2, 4}; // target = 2
    int target = 2;
    int display = fun(arr, 0, target); // SCp(display);
}
    
```

```

int fun(int[] arr, index, target) {
    
```

```

    if (index == arr.length) {
        return 0;
    }
    
```

```

    if (arr[index] == target) {
        return index;
    }
    
```

```

    fun(arr, index + 1, target);
}
    
```

O/p

// 1

dry run :-

O/p: 1

```

int fun(int[] arr, int index, int target) {
    
```

```

    if (index == arr.length) {
        return 0;
    }
    
```

```

    if (arr[index] == target) {
        return index;
    }
    
```

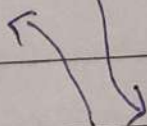
```

    fun(arr, index + 1, target);
}
    
```

~~F(0, 2)~~

~~F(1, 2)~~

call stack



```
if(arr[index] == number){  
    return index;  
}
```

```
return -1;  
}
```



```
int fun (int arr, int index, int target) {
```

```
    if (index == arr.length) {
```

```
        return 0;
```

```
    }
```

```
    if (arr[index] == target) {
```

```
        return index;
```

```
    }
```

```
}
```

Q2. Find last Occurrence (return index of last occurrence target)

→ psum () {

```
int arr = {5, 2, 3, 2, 4};
```

```
int target = 2;
```

```
S.O.p (fun(arr, 0, target));
```

```
}
```

```
int fun (int arr, int index, int target) {
```

```
    if (index == arr.length) {
```

```
        return -1;
```

```
    }
```

```
    int right = fun(arr, index + 1, target);
```

```
    if (right != -1) {
```

```
        return right;
```

```
    }
```

Q3

Count Occurrences. (count how many times length appears)

int countZero

→ psutil?

int arr = {1, 2, 2, 2, 3, 2, 2}

loop (fun(arr, 0, 2))

}

int fun(int arr, int index, int length)

if (index == arr.length)

return 0;

}


```

if (arr[index] == target) {
    return count + fun(arr, index + 1, target);
} else { fun(arr, index + 1, target); }
}

```

int fun (int () arr, int index, int target) {

if (index == arr.length) {

return 0

}

if (arr[index] == target) {

return count + fun(arr, index + 1, target);

} else { fun(); }

→ int fun (int () arr, int index, int target) {

if (index == arr.length) return 0;

if (arr[index] == target) {

return count + fun(arr, index + 1, target);

} else { fun(); }

int fun (int () arr, int index, int target) {

if (index == arr.length) return 0;

if (arr[index] == target) {

return count + fun(arr, index + 1, target);

} else { fun(); }

→ int fun (int () arr, int index, int target) {

if (index == arr.length) return 0;

if (arr[index] == target) {

return count + fun(arr, index + 1, target);

} else { fun(); }

Q. Check if Array is Sorted (return true if sorted in ascending order)

```

int prev = 0;
prev();

int arr = {1, 2, 3, 4} // op: true
Sorted fun(arr);

boolean fun(int arr, int index) {
    if (index == arr.length) {
        return true;
    }
    prev = arr[index];
    if (prev < arr[index+1]) {
        prev = arr[index+1];
        fun(arr, index+1);
        return true;
    } else {
        return false;
    }
}

```

```

// dry run
boolean fun(int arr, int index) {
    if (index == arr.length) {
        return true;
    }
    if (prev < arr[index+1]) {
        prev = arr[index+1];
        fun(arr, index+1);
    }
}

```

prev: 0, 1, 2, 3, 4

(1, 0) true
(2, 1)
(3, 2)
(4, 3)
(5, 4)
(6, 5)

```

boolean fun(int arr, int index) {
    if (index == arr.length) {
        return true;
    }
    if (prev < arr[index+1]) {
        prev = arr[index+1];
        fun(arr, index+1);
        return true;
    }
}

boolean fun(int arr, int index) {
    if (index == arr.length) {
        return true;
    }
    if (prev < arr[index+1]) {
        prev = arr[index+1];
        fun(arr, index+1);
        return true;
    }
}

```

```

// dry run
boolean fun(int arr, int index) {
    if (index == arr.length) {
        return true;
    }
    if (prev < arr[index+1]) {
        prev = arr[index+1];
        fun(arr, index+1);
    }
}

```

prev: 0, 1, 2, 3, 4

(1, 0) true
(2, 1)
(3, 2)
(4, 3)
(5, 4)
(6, 5)

Q5. Check if All element are even or odd
 -> pair() }
 int arr = {2, 4, 6}
 S.O.P (fun(arr, 0))

O/p: true

boolean fun(int[] arr, int index)

if (index == arr.length)

{

if (arr[index] % 2 == 0)

return true;

else

return false;

fun(arr, index + 1);

return true;

}

}

boolean fun(int[] arr, int index)

if (index == arr.length)

return -1;

if (arr[index] % 2 == 0)

return false;

else

fun(arr, index + 1);

O/p: true

f(1,4)
 f(1,3)
 f(1,2)
 f(1,1)
 f(1,0)

exit: k

boolean fun(int[] arr, int index) {
 if (index == arr.length) {
 return -1;
 }
 if (arr[index] % 2 == 0) {
 return false;
 }
 return true;
}

}

Q6 Check if Any Element is Negative

→ psum()

int() arr = {1, 2, 3, 4};
S.O.P (fun(arr, 0));

}

```
bool fun(int arr, int index) {
    if (index == arr.length)
        return -1;
}
```

```
if (arr[index] < 0) {
    return true;
}
```

```
return fun(arr, index + 1);
return false;
}
```

3

→ bool fun(int arr, int index) {

if (index == arr.length)

return -1;

if (arr[index] < 0)

return true;

}

fun(arr, index + 1);

}

f(1, 0)

f(1, 1)

f(1, 2)

call stack

bool fun(int arr, int index) {

if (index == arr.length)

return -1;

}

if (arr[index] < 0)

return true;

}

f(1, 0)

f(1, 1)

f(1, 2)

f(1, 3)

f(1, 4)

f(1, 5)

f(1, 6)

f(1, 7)

f(1, 8)

f(1, 9)

f(1, 10)

f(1, 11)

f(1, 12)

f(1, 13)

f(1, 14)

f(1, 15)

f(1, 16)

f(1, 17)

f(1, 18)

f(1, 19)

f(1, 20)

f(1, 21)

f(1, 22)

f(1, 23)

f(1, 24)

f(1, 25)

Q7. Find Maximum element in array

→ psum(1)

int max = 0;

int arr = {3, 9, 1, 7}

S.O.P (fun(arr, max, 0))

```
int fun(int l, arr, int index, int max) {
    if (index == arr.length) {
        return -1;
    }
    if (index < arr.length) {
        max = arr[index];
        return max;
    }
    return fun(arr, index + 1, max);
}
```

Edge case: int fun(int l, arr, int index, int max)

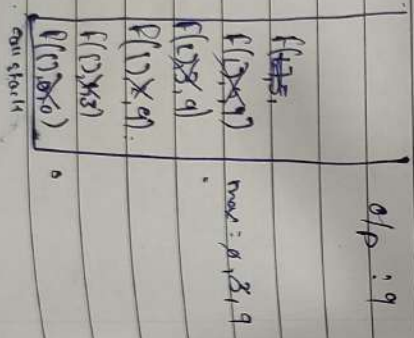
if (index == arr.length) {

return -1;

if (max < arr[index]) {

max = arr[index];

return fun(arr, index + 1, max);



int fun (int l, arr, int index, int max) {

if (index == arr.length) {

return -1;

if (max < arr[index]) {

max = arr[index];

return fun(arr, index + 1, max);

return max;

int fun (int l, arr, int index, int max)

if (index == arr.length) {

return -1;

}

f(0, 3, 9)

f(1, 9)

f(2, 1)

f(3, 7)

Q8. Find minimum element

→ psum () {

int min = 1

int arr = {3, 9, 1, 7}

S.O.P (dir, 0);

int fun(int arr, int index) {

if (index == arr.length - 1) {

return arr[index];

int min = fun(arr, index + 1);

if (min > arr[index]) {

return fun(arr, index + 1);

} else {

return min;

}

}

~~f(2, 4)~~

return 7

~~f(1, 3)~~

return 7

f(0, 2)

f(0, 1)

f(0, 0)

Q9 Sum of Elements at Odd index.

→
int sum = 0;
psum() {

int arr = {10, 20, 30, 40, 50};
s.o.p(fun(arr, 0));
}

int fun(int arr, int index) {

if (index == arr.length) {
return -1;

}

~~int sum = fun~~

if (arr[index] % 2 != 0) {
sum += arr[index];
}

fun(arr, index + 1);
}

Q10 Check palindrome array. (return true if array is palindrome)

bool

int arr[] = {1, 2, 3, 2, 1}

int original = arr;

S.O.P (fun(arr, index⁴, original⁰))

}

boolean fun(int arr, ^{int origindex} ~~int~~ original, int index)

if (index == ~~arr~~ length)

return true;

}

arr = fun(arr, index + 1);

original[origindex] = arr[index];

original++;

}

}

#dry run

(arr, 0, 0)

(arr, 0, 1)

original[0] = 1; orig++

(arr, 0, 2)

original[1] = 2; orig++

(arr, 0, 3)

original[2] = 3; orig++

(arr, 0, 4)

original[3] = 2; orig++

(arr, 0, 5)

original[4] = 1; orig++

F(0, 0, 5)

F(1, 0, 4)

F(2, 0, 3)

F(3, 0, 2)

F(4, 0, 1)

F(5, 0, 0)

call stack

YOUVA

~~Q12~~

Score = 15

3

4

3

#dry m

$$\max = 10$$

max: 5

fn $ax = 8$

~~max-70~~
$$= f(L, 4)$$

→ 511 may = 15

(check strictly increasing. (not even equal))

arr = {1, 2, 3, 3}

isSorted()

int arr = {1, 2, 3, 3}

S.O.P (fun(arr, 0))

}

int fun (int arr, int index) {

if (index == arr.length)

return true;

}

if (arr[index] < arr[index+1] && fun(arr, index+1))

fun(arr, index+1);

return true;

} else {

return false;

}

}

1 < 2 && 2 < 3 && 3 < 3 != 2

f(1, 0)

f(1, 1)

2 < 3 && 3 < 3 != 3

false

f(1, 2)

f(1, 3)

f(1, 4)

f(1, 5)

f(1, 6)

Count Clement Graham

111

6. 2019. 10. 10.

$$0/h = 2$$

Willard, W. H.

3

relation $f(x)$ is

6700

177

100

Count:

0

$\tau = 1$

3

$\psi(1, 3)$

 $\Gamma(8,4) \rightarrow \dots$

Beck's Array Contains Target

2

$$\text{the } q \text{th } r_{q+3}$$

1

Figure - 1

2. $\frac{d}{dx} \ln(x) = \frac{1}{x}$

2

Wavelength

10

free

156, 411

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----

1



Q15 Find sum of All Negative numbers

return 0;

d/p = 6

int arr = {1, -2, 3, -4, 5};

S.O.P (fun(arr, 0));

int sum = 0;

int fun(int arr, int index) {

if (index == arr.length) {

return -3; // -2

if (arr[index] < 0) {

sum += arr[index];

fun(arr, index+1);

return sum; }

f(7, 0)

f(7, 1)

sum = 3

f(7, 2)

f(7, 3)

sum = -6

f(7, 4)

sum = -6

f(7, 5)

Q16 Find index of smallest element

return 0;

int arr = {5, 2, 8, 1, 3};

S.O.P (fun(arr, 0));

int smIndex;

int fun(int arr, int index) {

if (index == arr.length) {

return -1;

if (arr[index] < smIndex) {

smIndex = arr[index];

fun(arr, index+1);

return smIndex; }

f(7, 0)

smIndex = 5

f(7, 1)

smIndex = 2

f(7, 2)

smIndex = 1

f(7, 3)

f(7, 4)

f(7, 5)

Q17) Count element divisible by k.

→ Example

arr = [2, 4, 5, 6, 9]

int k = 2.

S.O.P (fun(arr, 0, k)):

int count = 0;

int fun(int arr, int index, int k) {

if (index == arr.length) {

return 0;

if (arr[index] % k == 0) {

count++;

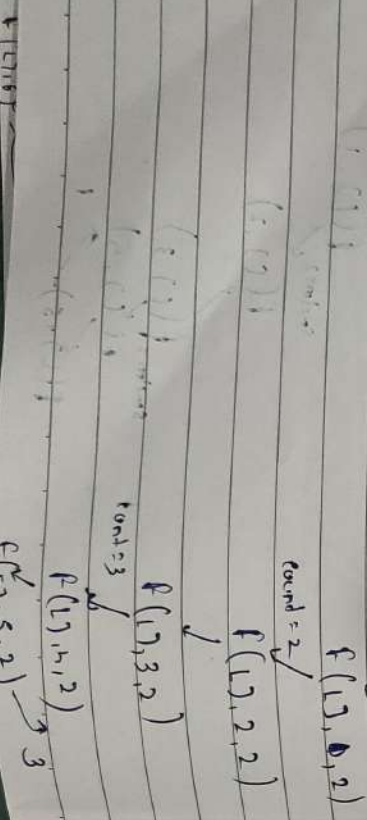
}

return fun(arr, index + 1, k);

return count;

}

Hydro



Q18) Check if an element is unique or not.

→ Example

arr = [1, 2, 3, 5]

3

S.O.P (fun(arr, 0, k)):

boolean fun (int arr, int index) {

if (index == arr.length) {

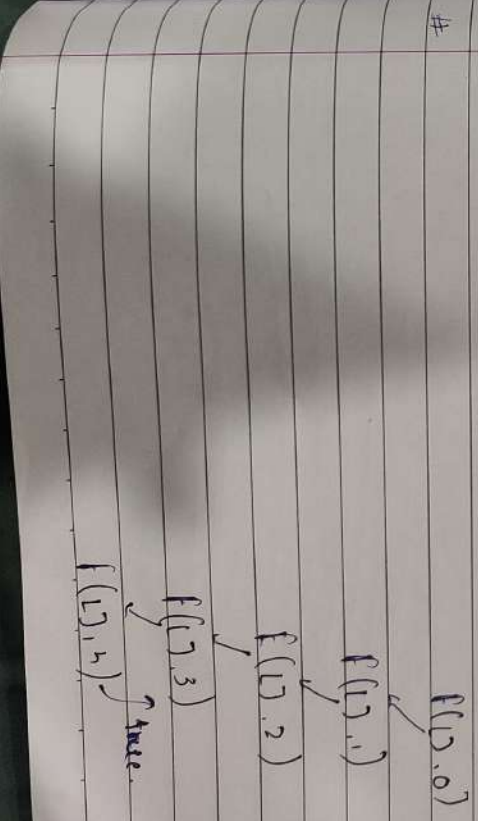
return true;

if (arr[index] == arr[index + 1]) {

return false;

return fun(arr, index + 1);

}



Q19 Find sum of length of all elements in an array

para 11}

int l) arr = [12, 34, 5]

S.O.P (fun(arr, 0))

}

int sum = 0;
int fun(int l, arr, int index) {

if (index == arr.length - 1)

return arr[index];

while (arr[index] > 0) {

sum += arr[index];

return sum;

fun(arr, index + 1);

return sum;

}

(0, 1) {

(1, 2) {

(2, 3) {

(3, 4) {

(4, 5) {

Q20 Check if array is strictly decreasing.

para 11}

int l) arr = [9, 7, 8, 2]

S.O.P (fun(arr, 0))

}

boolean fun(int l, arr, int index) {

if (index == arr.length - 1)

return false;

}

if (arr[index] > arr[index + 1]) {

fun(arr, index + 1);

return true;

}

return false;

}